

Advanced Optimization

Chapter 1 — Introduction

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 1

Where this course sits

Convex Optimization (prerequisite)

- ▶ *What can be solved?*
- ▶ Modeling, convex sets and functions
- ▶ Duality, KKT conditions
- ▶ Guarantee: local = global

This course

- ▶ *How is it actually solved?*
- ▶ Algorithms and their mechanics
- ▶ Nonconvexity is the default
- ▶ Guarantee: stationarity, at best

Destination of the semester: **Chapter 19, interior-point methods for nonlinear programming**
— what actually runs inside IPOPT or KNITRO when you solve an AC-OPF.

Two questions this course answers

Q1 If we give up global optimality, **what can we still guarantee?**

- Convergence to a stationary point — from where, and how fast
- What happens when the Hessian is indefinite
- What happens when constraints cannot all be satisfied at once

Q2 Why do production solvers look the way they do?

- Why Wolfe conditions, not just “decrease f ”
- Why a filter instead of a merit function
- Why a “restoration phase” exists at all

Style of this course

Convergence theorems will be stated and used, almost never proved. Derivations that *define* an algorithm (CG, BFGS, SQP) will be done in full; the rest go to homework.

The three ingredients, and the general form

- ▶ $x \in \mathbb{R}^n$ — the **variables** (unknowns, parameters)
- ▶ $f : \mathbb{R}^n \rightarrow \mathbb{R}$ — the **objective**, a single scalar measure of performance
- ▶ $c_i : \mathbb{R}^n \rightarrow \mathbb{R}$ — the **constraint functions**, equalities and inequalities x must satisfy

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad \begin{cases} c_i(x) = 0, & i \in \mathcal{E}, \\ c_i(x) \geq 0, & i \in \mathcal{I}. \end{cases} \quad (1.1)$$

Sign convention — different from Boyd

Nocedal & Wright write inequalities as $c_i(x) \geq 0$. Boyd writes $f_i(x) \leq 0$. Same content, opposite sign. This flips the sign of Lagrange multipliers in the KKT conditions later (Ch. 12) — keep it in mind when you compare the two books.

Identifying f , x , and c_i for a real problem is *modeling*. This course does not treat modeling; we take the problem as given and ask how to solve it.

A vocabulary for classifying problems

Every optimization problem can be located on several axes at once.

Axis	This course	Elsewhere
Continuous / Discrete	continuous	Discrete Optimization
Unconstrained / Constrained	both (Ch. 2–11 / 12–19)	—
Local / Global	local	global optimization
Deterministic / Stochastic	deterministic	stochastic programming, SGD
Convex / Nonconvex	nonconvex	Convex Optimization
Smooth / Nonsmooth	smooth (C^2)	subgradient methods
Small / Large scale	both, emphasis on large	—

The next few slides say one thing about each axis — mostly to mark *where the boundary of this course lies*.

Continuous vs. discrete

- ▶ **Discrete:** x is drawn from a finite (but often enormous) set — integers, binaries, permutations. Integer programming, MIP.
- ▶ **Continuous:** the feasible set is uncountable; $x \in \mathbb{R}^n$.
- ▶ Why continuous is “easier”: **smoothness lets information at x tell you about f near x** . In discrete problems, neighbors can behave arbitrarily differently.
- ▶ Discrete feasible sets are, in a sense, an extreme form of nonconvexity: a convex combination of two feasible points is generally infeasible.

The two courses are not disjoint

Branch-and-bound solves a sequence of continuous *relaxations*. Every node of the tree runs a continuous solver. What you learn here is what runs inside the discrete methods you already know.

Unconstrained vs. constrained

- ▶ **Unconstrained:** $\mathcal{E} = \mathcal{I} = \emptyset$. Arises directly, or when constraints are inactive and can safely be ignored.
- ▶ **Constrained:** constraints are essential — budgets, physical limits, power balance. Range from simple bounds $0 \leq x_1 \leq 100$ to nonlinear inequalities.
- ▶ **Linear programming:** everything linear. Probably the most widely solved class.
- ▶ **Nonlinear programming (NLP):** at least one nonlinear objective or constraint. Native to the physical sciences and engineering — and to AC-OPF.

The bridge between the two halves

Constrained problems are often *reformulated* as unconstrained ones by adding penalty terms that discourage violation. This is why the unconstrained half of the course is not optional: it is the engine inside the constrained half (Ch. 17).

Local vs. global

- ▶ **Local solution:** no feasible nearby point does better.
- ▶ **Global solution:** no feasible point anywhere does better.
- ▶ For convex problems (and LPs in particular) the two coincide.
- ▶ For general nonlinear problems they need not — and **global solutions are hard to recognize, harder to locate.**

The identity of this course

We compute and characterize *local* solutions. Global optimization is treated only in passing. This is not a weakness to apologize for: most successful global methods work by solving many local problems, using exactly the algorithms in this book.

Deterministic vs. stochastic

- ▶ **Deterministic:** the model is completely known. This book.
- ▶ **Stochastic:** some data are uncertain at formulation time — future demand, prices, renewable output. Optimize *expected* performance over scenarios.
- ▶ Related paradigms worth knowing by name:
 - *chance-constrained* — constraints hold with prescribed probability
 - *robust* — constraints hold for all values of the uncertain data

Where it reconnects

Stochastic methods typically solve deterministic subproblems, so the techniques here still apply. And the “stochastic” in *stochastic gradient descent* is a different beast altogether — we will read Bottou, Curtis & Nocedal (2018) in seminar to see how it fits into this semester’s vocabulary.

Convexity — a greeting to the world we are leaving

Recall: S is convex if it contains every segment between its points; f is convex if $f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y)$ for $\alpha \in [0, 1]$.

Convex programming in the notation (1.1):

- ▶ objective f convex,
- ▶ equality constraints $c_i, i \in \mathcal{E}$, *linear*,
- ▶ inequality constraints $c_i, i \in \mathcal{I}$, *concave* (because $c_i \geq 0$, not ≤ 0 — see the sign remark).

What convexity buys, in one sentence

Any local solution is a global solution.

Everything in the prerequisite course rested on that sentence. **From here on, we do not assume it.** Convexity will reappear only as a *property of subproblems* — the QP inside SQP, the barrier problem inside IPM — never as a property of the problem we are actually solving.

Smooth vs. nonsmooth; small vs. large

Smoothness

This course: f and c_i are C^2 — second derivatives exist and are continuous.

Nonsmooth optimization (subgradients) is out of scope. But note: some nonsmooth objectives such as $\|r(x)\|_1$ or $\|r(x)\|_\infty$ can be rewritten as smooth *constrained* problems — and thereby brought back in.

Scale

The book, and this course, emphasize large-scale techniques: limited-memory quasi-Newton, inexact Newton, interior-point, automatic differentiation. “Large” means: you cannot store or factor an $n \times n$ dense matrix. For a 4,500-bus grid this is already the regime you live in.

Optimization algorithms are iterative

Start from an initial guess x_0 ; generate iterates $x_1, x_2, \dots$; stop when no further progress is possible or a solution is approximated well enough.

What distinguishes algorithms is *the rule for moving from x_k to x_{k+1}* : which information is used (f , ∇f , $\nabla^2 f$, constraints), and whether information from earlier iterates is remembered.

Three properties a good algorithm should have

- ▶ **Robustness** — works on a wide class of problems, from any reasonable x_0
- ▶ **Efficiency** — modest time and storage
- ▶ **Accuracy** — reaches the solution precisely, tolerates data and rounding error

These conflict. A fast-converging method may need too much storage; the most robust method may be the slowest. Managing these trade-offs *is* numerical optimization, and is what the rest of the semester is about.

The role of theory in this course

The book's own position, which we adopt:

Theory does two jobs — it characterizes optimal points (optimality conditions), and it provides the basis for the algorithms. One cannot understand numerical optimization without it.

What that means operationally here:

- ▶ Optimality conditions: we state them, derive the simple ones, and use them constantly.
- ▶ Convergence analysis: we state the results and read them for their *assumptions* — where does the guarantee break, and what does the algorithm do about it.
- ▶ Proofs: assigned as homework where the derivation is instructive; skipped otherwise.

Semester roadmap

Week	Ch.	Topic
1	1–2	Framing; optimality conditions; overview of algorithms
2	3	Line search: Wolfe conditions, backtracking, Hessian modification
3	4	Trust region: Cauchy point, dogleg, Steihaug-CG
4	5	Conjugate gradient: conjugacy, preconditioning, nonlinear CG
5	6	Quasi-Newton: secant equation, BFGS, SR1
6	7	Large-scale: L-BFGS, inexact Newton-CG
7	8	Derivatives: finite differences, automatic differentiation, adjoint
8	—	<i>Midterm</i>
9	10–11	Least squares and nonlinear equations: Gauss–Newton, LM, Newton–Raphson
10	12	Constrained theory: KKT revisited, constraint qualifications
11	16	Quadratic programming: the KKT system, active-set methods
12	17	Penalty and augmented Lagrangian methods
13	18	Sequential quadratic programming
14	19	Interior-point methods I: primal-dual system, barrier update
15	19	Interior-point methods II: nonconvexity, filter, restoration — IPOPT dissected
16	—	<i>Final</i>

Not covered, and why

- ▶ **Ch. 9** Derivative-free optimization — different problem class.
- ▶ **Ch. 13–14** Simplex and LP interior-point — covered in Discrete Optimization.
- ▶ **Nonsmooth, global, stochastic, discrete** — see the classification slide; each is its own field.
- ▶ **Most convergence proofs** — stated, used, assigned, not lectured.

Next: Chapter 2

Before any algorithm — what does “solution” mean once we drop convexity, and how do we recognize one from local information alone?